Experiment 7 : (single linked list)

Q1) Write a C program to perform the operations on a single linked list:

i) Insertion at beginning, ii) Deletion of 1st node iii) display all nodes

```
#include<stdio.h>
#include<stdlib.h>
struct node{
   int data;
   struct node *next;
};
struct node *head=NULL;
void insert_begin(){
   struct node *temp;
   int x;
   temp=(struct node*)malloc(sizeof(struct node));
   printf("Enter value: ");
   scanf("%d",&x);
   temp->data=x;
   temp->next=head;
   head=temp;
}
void delete_first(){
   struct node *temp;
   if(head==NULL){
      printf("List empty\n");
      return;
   }
   temp=head;
   head=head->next;
```

```
    free(temp);
}
void display(){
    struct node *temp=head;
    while(temp!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("\n");
}
int main(){
    int ch;
    while(1){
        printf("1.Insert Beginning 2.Delete First 3.Display 4.Exit\n");
        scanf("%d",&ch);
        switch(ch){
            case 1: insert_begin(); break;
            case 2: delete_first(); break;
            case 3: display(); break;
            case 4: exit(0);
        }
    }
}
```

Q2) Write a C program to perform the operations on a single linked list:

i) insertion at end, i) deletion of last node iii) display all the nodes

```
#include<stdio.h>
#include<stdlib.h>
struct node{
   int data;
   struct node *next;
};
struct node *head=NULL;
void insert_end(){
   struct node *temp,*ptr;
   int x;
   temp=(struct node*)malloc(sizeof(struct node));
   printf("Enter value: ");
   scanf("%d",&x);
   temp->data=x;
   temp->next=NULL;
   if(head==NULL){
      head=temp;
      return;
   }
   ptr=head;
   while(ptr->next!=NULL)
      ptr=ptr->next;
   ptr->next=temp;
}
void delete_last(){
   struct node *temp=head,*prev;
```

```
    if(head==NULL){
        printf("List empty\n");
        return;
    }
    if(head->next==NULL){
        free(head);
        head=NULL;
        return;
    }
    while(temp->next!=NULL){
        prev=temp;
        temp=temp->next;
    }
    prev->next=NULL;
    free(temp);
}
void display(){
    struct node *temp=head;
    while(temp!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("\n");
}
int main(){
    int ch;
    while(1){
        printf("1.Insert End 2.Delete Last 3.Display 4.Exit\n");
```

```
        scanf("%d",&ch);
        switch(ch){
            case 1: insert_end(); break;
            case 2: delete_last(); break;
            case 3: display(); break;
            case 4: exit(0);
        }
    }
}
```

Q3) Write a C program to perform the operations on a single linked list:

i) insertion at location ii) searching for a node item iii) display all the nodes.

```
#include<stdio.h>

#include<stdlib.h>

struct node{

   int data;

   struct node *next;

};

struct node *head=NULL;

void insert_pos(){

   struct node *temp,*ptr;

   int x,pos,i;

   printf("Enter value and position: ");

   scanf("%d%d",&x,&pos);

   temp=(struct node*)malloc(sizeof(struct node));

   temp->data=x;

   if(pos==1){

      temp->next=head;

      head=temp;

      return;

   }

   ptr=head;

   for(i=1;i<pos-1 && ptr!=NULL;i++)

      ptr=ptr->next;

   if(ptr==NULL){

      printf("Invalid position\n");

      return;
```

```
    }
    temp->next=ptr->next;
    ptr->next=temp;
}
void search(){
    struct node *temp=head;
    int key,pos=1;
    printf("Enter value to search: ");
    scanf("%d",&key);
    while(temp!=NULL){
        if(temp->data==key){
            printf("Found at position %d\n",pos);
            return;
        }
        temp=temp->next;
        pos++;
    }
    printf("Not found\n");
}
void display(){
    struct node *temp=head;
    while(temp!=NULL){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("\n");
}
```

```
int main(){
    int ch;
    while(1){
        printf("1.Insert Position 2.Search 3.Display 4.Exit\n");
        scanf("%d",&ch);
        switch(ch){
            case 1: insert_pos(); break;
            case 2: search(); break;
            case 3: display(); break;
            case 4: exit(0);
        }
    }
}
```